

Exercice 1 : Conception d'un compteur électronique

En électronique, un compteur est un composant essentiel. Considérons ici un **compteur 8 bits** comportant :

- Une entrée d'activation **synchrone** appelée **Enable**.
- Un signal de **reset asynchrone** nommé **rst**.
- Une incrémentation qui s'effectue sur les fronts montants de l'horloge **clk**.

Questions :

- 1) Combien de bascules sont nécessaires pour implémenter ce compteur ?
8 bascules, chaque bit étant représenté par une bascule.
- 2) Dessinez un chronogramme illustrant le fonctionnement du compteur.
Ci-dessous, un exemple de simulation avec Vivado pour un compteur utilisant une horloge d'une période de 10 ns.

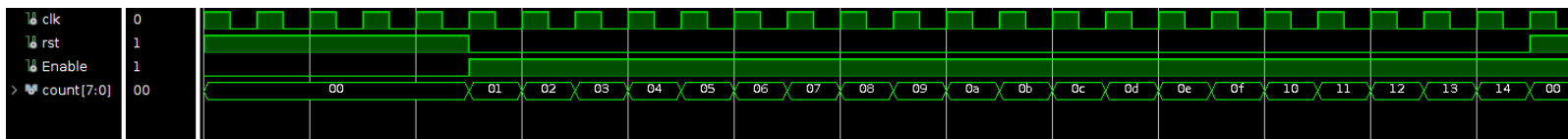


Figure -1

- 3) Écrivez le code de ce compteur en **VHDL** et en **Verilog**.

Vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
entity compteur is
    Port (
        clk : in STD_LOGIC; --horloge
        rst : in STD_LOGIC; --reset
        Enable : in STD_LOGIC; --activation
        count : out STD_LOGIC_VECTOR (7 downto 0) -- sortie sur 8 bits
    );
end compteur;
architecture Behavioral of compteur is
    signal count_internal : unsigned(7 downto 0) := (others => '0');
begin
    begin
        process (clk, rst)
        begin
            if rst = '1' then
                count_internal <= (others => '0');
            elsif rising_edge(clk) then
                if Enable = '1' then
                    count_internal <= count_internal + 1;
                end if;
            end if;
        end process;
        count <= std_logic_vector(count_internal);
    end Behavioral;
```

Verilog

```

module compteur
( input wire clk,
  input wire rst,
  input wire Enable,
  output reg [7:0] Count );
always @(posedge clk or posedge rst) begin
    if (rst) begin
        Count <= 8'b0;
    end else if (Enable)
        begin
            Count <= Count + 1;
        end
    end
end
endmodule

```

4) Créer le TestBench pour le compteur en VHDL :

Le TestBench permet de vérifier le bon fonctionnement du système. Dans l'exemple suivant, on teste la réaction du système à une horloge avec une période de 10 ns. Le signal reset et le signal enable sont respectivement à 1 et 0 pendant 50 ns. Ensuite, reset passe à 0 et enable à 1 pendant 200 ns. Enfin, les deux signaux sont positionnés à 1, (**Figure -1**).

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```

entity compteur_tb is
end compteur_tb;
architecture Behavioral of compteur_tb is

```

```

    component compteur
    Port (
        clk   : in  STD_LOGIC;
        rst    : in  STD_LOGIC;
        Enable : in  STD_LOGIC;
        count   : out STD_LOGIC_VECTOR (7 downto 0)
    );
end component;

```

```

signal clk   : STD_LOGIC := '0';
signal rst    : STD_LOGIC := '0';
signal Enable : STD_LOGIC := '0';
signal count  : STD_LOGIC_VECTOR (7 downto 0);

```

```

begin
    uut: compteur
    Port map (
        clk   => clk,
        rst    => rst,
        Enable => Enable,
        count  => count
    );
    clk_process : process
    begin
        clk <= not(clk);
        wait for 5 ns;
    end process;

    stim_proc: process
    begin
        rst <= '1';
        Enable <= '0';
        wait for 50 ns;
        rst <= '0';
        Enable <= '1';
    end process;

```

```
    wait for 200 ns;
    Enable <= '1';
    rst <= '1';
    wait;
end process;
end Behavioral;
```

4) Créer le TestBench pour le compteur en Verilog :

```
`timescale 1ns / 1ps
module tb_compteur();

    reg clk = 1;
    reg rst = 0;
    reg Enable = 0;
    wire [7:0] Count;

    compteur uut (
        .clk(clk),
        .rst(rst),
        .Enable(Enable),
        .Count(Count)
    );

    initial begin
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        Enable = 0;

        #50 rst = 0;
        Enable = 1;

        #200 rst = 1;
        Enable = 1;
    end
endmodule
```

Amélioration :

Nous souhaitons maintenant étendre le fonctionnement du compteur avec deux nouvelles entrées :

- **up** : Si active, le compteur s'incrémente.
- **down** : Si active, le compteur se décrémente.

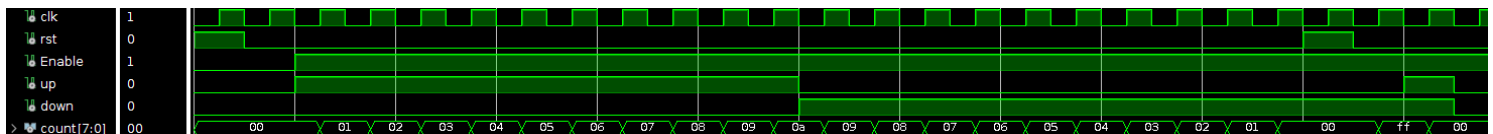
Mise à jour demandée :

4. Modifiez le code VHDL pour intégrer cette nouvelle fonctionnalité.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity compteur_ext is
  Port (
    clk  : in  STD_LOGIC;
    rst  : in  STD_LOGIC;
    Enable : in  STD_LOGIC;
    up   : in  STD_LOGIC;
    down : in  STD_LOGIC;
    count : out STD_LOGIC_VECTOR (7 downto 0)
  );
end compteur_ext;

architecture Behavioral of compteur_ext is
  signal count_internal : unsigned (7 downto 0) := (others => '0');
begin
  process (clk, rst)
  begin
    if rst = '1' then
      count_internal <= (others => '0'); -- Réinitialisation du compteur
    elsif rising_edge(clk) then
      if Enable = '1' then
        if up = '1' and down = '0' then
          count_internal <= count_internal + 1; -- Incrémentation
        elsif down = '1' and up = '0' then
          count_internal <= count_internal - 1; -- Décrémentation
        else count_internal <= (others => '0'); --si up et down sont actifs en meme temps on aura 0
        end if;
      end if;
    end if;
  end process;
  count <= std_logic_vector(count_internal); -- Conversion en std_logic_vector pour la sortie
end Behavioral;
```

Voici un exemple de simulation

Exercice 2 : Conception d'un registre à décalage SIPO de N bits

1) On dispose de la description VHDL d'une bascule D (flip-flop) dont l'entité est donnée ci-dessus. Ecrivez le testbench de ce composant

```
entity bascule is
port(d, clk, reset, enable : in std_logic ;
q : out std_logic);
end entity bascule;
```

➔ Voici un TestBench conçu pour tester le système. L'objectif est de vérifier un maximum d'entrées, voire toutes si possible, afin d'observer la réaction de la sortie et de s'assurer qu'elle correspond aux attentes.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity tb_bascule is
end tb_bascule;
```

```
architecture bhv of tb_bascule is
  signal tb_clk : std_logic := '0';
  signal tb_d : std_logic := '0';
  signal tb_enable : std_logic := '0';
  signal tb_reset : std_logic := '0';
  signal tb_q : std_logic;
```

```
  component bascule
  port (
    d, clk, rst, enable : in std_logic;
    q : out std_logic
  );
end component;
```

```
begin
  i1: bascule
  port map (
    d => tb_d,
    clk => tb_clk,
    rst => tb_reset,
    enable => tb_enable,
    q => tb_q
  );
```

tb_clk <= not tb_clk after 5 ns; -- Il est possible de tester le signal d'horloge (**clk**) sans utiliser de processus.

```
stim_proc: process
begin
  tb_reset <= '1';
  tb_enable <= '0';
  tb_d <= '0';
  wait for 10 ns;
  tb_reset <= '0';
  wait for 10 ns;
  tb_d <= '1';
  wait for 20 ns;
  tb_enable <= '1';
  wait for 10 ns;
  tb_d <= '0';
  wait for 20 ns;
  tb_reset <= '1';
```

SN360/SN361 : Introduction à la conception des circuits numériques

```

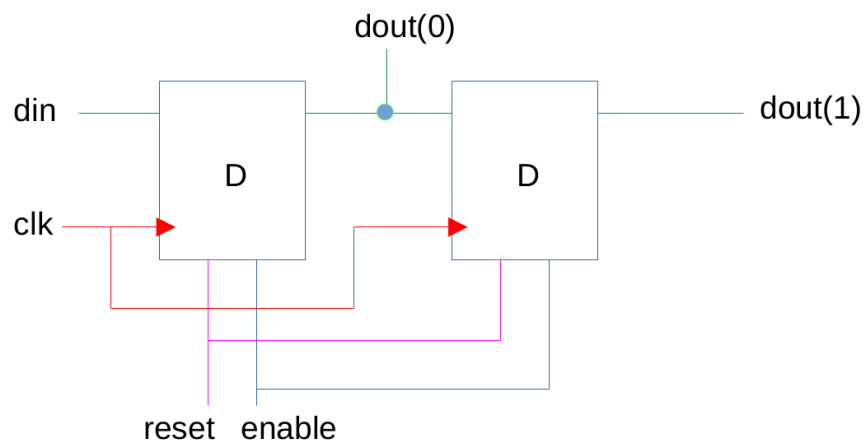
wait for 10 ns;
tb_reset <= '0';
wait for 20 ns;
wait;
end process;
end bhv;

```

2. On veut construire un registre à décalage de 2 bits SIPO (Serial In – Parallel Out) en utilisant uniquement 2 bascules D. Ce registre à décalage comporte :

- un simple bit d'entrée (din)
- une entrée d'autorisation de décalage (enable) active à l'état haut
- une sortie parallèle (dout) sur 2 bits
- une remise à zéro (reset) synchrone active à l'état haut.

2.1. Dessinez le schéma de ce registre



2.2. Ecrivez la description structurelle du registre SIPO (entité + architecture)

```

library IEEE;
use IEEE. std_logic_1164.all;

```

```

entity SIPO is
    port( din, enable, clk, reset : in std_logic;
          dout : out std_logic_vector(0 to 1));
end SIPO;

```

architecture struct of SIPO is

```

component bascule
    port(d, clk, reset, enable : in std_logic ;
          q : out std_logic);
end component;

```

```

signal dout_temp : std_logic_vector(0 to 1);
begin

```

```

i1: bascule
    port map(d => din , enable => enable , clk => clk, reset => reset, q => dout_temp(0));

```

```

i2: bascule
    port map (d => dout_temp(0), enable => enable, clk => clk, reset => reset, q => dout_temp(1));
    dout<=dout_temp ;

```

```

end struct;

```

SN360/SN361 : Introduction à la conception des circuits numériques

Pour que le registre soit facilement réutilisable dans différents projets, on souhaite le rendre plus générique. On veut qu'il soit paramétrable pour supporter différentes tailles de vecteurs de sortie. On veut donc construire un registre à décalage SIPO de N bits.

3. Ecrivez l'entité de ce registre

```
library IEEE;
use IEEE.std_logic_1164.all;

entity SIPO_n is
  generic (N : integer := 5);
  port (
    din   : in std_logic;
    enable : in std_logic;
    clk   : in std_logic;
    reset  : in std_logic;
    dout  : out std_logic_vector(0 to N-1)
  );
end SIPO_n;

architecture struct of SIPO_n is

  component bascule
    port (
      d    : in std_logic;
      clk  : in std_logic;
      reset : in std_logic;
      enable : in std_logic;
      q    : out std_logic
    );
  end component;
  signal dout_temp : std_logic_vector(0 to N);

begin

  bascules_gen : for I in 0 to N-1 generate
  begin
    bascule_int: bascule
      port map (
        d    => dout_temp(I),
        enable => enable,
        clk  => clk,
        reset => reset,
        q    => dout_temp(I+1)
      );
  end generate;
  dout_temp(0) <= din;
  dout <= dout_temp(1 to N);

end architecture;
```

TestBench

```
library IEEE;
use IEEE. std_logic_1164.all;

entity tb_SIPO_n is
end tb_SIPO_n;

architecture bhv of tb_SIPO_n is
  signal tb_clk : std_logic := '1';
  signal tb_din, tb_enable, tb_reset: std_logic;
  signal tb_dout: std_logic_vector(0 to 4);

  component SIPO_n
    generic(N : integer :=5);
    port( din, enable, clk, reset : in std_logic;
          dout : out std_logic_vector(0 to N-1));
  end component;

begin

  i1:SIPO_n
    generic map(N=>5)
    port map (din=> tb_din,enable => tb_enable, clk => tb_clk,reset => tb_reset,dout => tb_dout) ;

  tb_clk <= not(tb_clk) after 5 ns;

  tb_din <= '0', '1' after 5 ns, '0' after 25 ns, '1' after 45 ns;
  tb_enable <= '0', '1' after 15 ns, '0' after 25 ns, '1' after 35 ns;
  tb_reset <= '0', '1' after 55 ns, '0' after 65 ns;

end bhv;
```